
title: Task Manager JWT — Documentación Técnica Completa author: Euclides Marín

date: Mayo 2026

Task Manager JWT

Documentación Técnica Completa

Sistema fullstack de gestión de tareas con autenticación JWT

Stack: PostgreSQL · Express.js · React · Node.js (PERN)

Tabla de Contenidos

1. [Descripción General](#)
2. [Arquitectura del Sistema](#)
3. [Stack Tecnológico](#)
4. [Estructura del Proyecto](#)
5. [Base de Datos](#)
6. [Backend — Paso a Paso](#)
 - 6.1 Punto de Entrada: server.js
 - 6.2 Configuración de Base de Datos
 - 6.3 Utilidades JWT
 - 6.4 Schemas de Validación (Zod)
 - 6.5 Middleware de Autenticación
 - 6.6 Middleware de Roles
 - 6.7 Manejo Centralizado de Errores
 - 6.8 Controlador de Autenticación
 - 6.9 Controlador de Tareas
 - 6.10 Controlador de Administración
 - 6.11 Rutas
7. [Frontend — Paso a Paso](#)
 - 7.1 Entrada de la Aplicación
 - 7.2 Redux Store
 - 7.3 Axios con Interceptores
 - 7.4 Slice de Autenticación
 - 7.5 Slice de Tareas
 - 7.6 Componente PrivateRoute
 - 7.7 Páginas
8. [Flujo de Autenticación JWT](#)

- 9. [API Reference](#)
- 10. [Variables de Entorno](#)
- 11. [Instalación y Puesta en Marcha](#)
- 12. [Docker — Contenedorización](#)
- 13. [Tests](#)
- 14. [Deploy con Docker y Nginx Proxy Manager](#)
- 15. [Seguridad](#)

1. Descripción General

Task Manager JWT es una aplicación web fullstack que permite a los usuarios gestionar sus tareas personales con autenticación segura basada en JSON Web Tokens.

Funcionalidades principales

Funcionalidad	Descripción
Registro e inicio de sesión	JWT con access token (15 min) + refresh token (7 días)
CRUD de tareas	Crear, leer, actualizar y eliminar tareas propias
Filtros y búsqueda	Por estado, prioridad, fechas y texto libre
Paginación	Resultados paginados con page y limit
Exportación CSV	Descarga de todas las tareas en formato CSV
Recuperación de contraseña	Cambio directo de contraseña o via token por email
Panel de administración	Visibilidad total sobre usuarios y tareas (solo admin)
Persistencia de sesión	Renovación automática del access token vía refresh token

2. Arquitectura del Sistema

· Axios		Cliente (React + Vite)	Redux · React Router
<accessToken>		HTTP / REST	Authorization: Bearer
Cookie: refreshToken (HTTP-Only)			
CORS · Rate Limit		Backend (Express + Node.js)	Helmet ·
Prisma ORM		JWT · Zod · Bcrypt	
users · tasks		PostgreSQL 16	tablas:

Arquitectura Docker (producción)

Internet Nginx Proxy Manager (red: proxy_network) task-frontend :80 (Nginx Alpine + build Vite) task-backend :3000 (Node 20 Alpine) task_internal_net (red privada bridge) task-db :5432 (PostgreSQL 16 Alpine)

Flujo de una petición autenticada

```

- 1. Usuario hace acción en React
- 2. Axios adjunta el accessToken (header Bearer)
- 3. Express verifica el token en authMiddleware
- 4. Si válido   controller ejecuta la lógica
- 5. Controller consulta/escribe en PostgreSQL via Prisma
- 6. Respuesta JSON regresa al cliente
- 7. Redux actualiza el estado global
- 8. React re-renderiza el componente ```

---

## 3. Stack Tecnológico

### Backend

| Paquete            | Versión | Rol                                          |
|--------------------|---------|----------------------------------------------|
| express            | ^4.21   | Servidor HTTP y routing                      |
| @prisma/client     | ^5.22   | ORM — interfaz con PostgreSQL                |
| prisma             | ^5.22   | CLI de migraciones                           |
| jsonwebtoken       | ^9.x    | Generación y verificación de JWT             |
| bcrypt             | ^5.x    | Hash seguro de contraseñas                   |
| zod                | ^3.x    | Validación de datos en runtime               |
| helmet             | ^7.x    | Headers de seguridad HTTP                    |
| cors               | ^2.x    | Control de políticas CORS                    |
| cookie-parser      | ^1.x    | Lectura de cookies HTTP                      |
| express-rate-limit | ^7.x    | Protección contra fuerza bruta               |
| dotenv             | ^16.x   | Variables de entorno                         |
| nodemailer         | ^6.x    | Envío de emails (recuperación de contraseña) |

## Frontend

| Paquete             | Versión | Rol                                     |
|---------------------|---------|-----------------------------------------|
| react               | ^18.x   | Librería de interfaz de usuario         |
| vite                | ^8.x    | Bundler y servidor de desarrollo        |
| @reduxjs/toolkit    | ^2.x    | Gestión de estado global                |
| react-redux         | ^9.x    | Integración Redux con React             |
| react-router-dom    | ^6.x    | Enrutamiento client-side (SPA)          |
| axios               | ^1.x    | Cliente HTTP con interceptores          |
| react-hook-form     | ^7.x    | Manejo de formularios                   |
| zod                 | ^3.x    | Validación de formularios               |
| @hookform/resolvers | ^3.x    | Integración Zod + React Hook Form       |
| tailwindcss         | ^3.x    | Estilos utilitarios (utility-first CSS) |

---

## 4. Estructura del Proyecto

task-manager-jwt/      docker-compose.yml      Orquestación de contenedores  
    .env      Variables de entorno para Docker      backend/      Dockerfile  
Imagen Node 20 Alpine      entrypoint.sh      Migraciones + arranque del servidor  
    server.js      Punto de entrada      .env / .env.example      package.json  
prisma/      schema.prisma      Modelos de BD      migrations/      Historial de  
migraciones      src/      config/      db.js      Instancia Prisma  
controllers/      authController.js      register, login, refresh, logout,  
forgotPassword, resetPassword      taskController.js      CRUD tareas +  
exportCSV      adminController.js      getUsers, getUserTasks, updateRole  
middleware/      authMiddleware.js      Verificación JWT  
roleMiddleware.js      Control de roles      errorHandler.js      Manejo central de  
errores      routes/      authRoutes.js      taskRoutes.js  
adminRoutes.js      schemas/      authSchemas.js      Zod: register, login,  
forgotPassword, resetPassword      taskSchemas.js      Zod: createTask,  
updateTask      utils/      jwt.js      Generación/verificación tokens  
csvExporter.js      Generación de CSV      tests/      auth.test.js      tasks.test.js  
    frontend/      Dockerfile      Build multietapa (Node Nginx)      nginx.conf  
Config Nginx para SPA y caché      index.html      HTML raíz (Vite)      vite.config.js  
tailwind.config.js      .env / .env.example      package.json      src/      main.jsx  
Punto de entrada React      App.jsx      Rutas principales      index.css      Estilos  
globales + Tailwind      app/      store.js      Redux store      features/      auth/  
    authSlice.js      Estado + thunks de auth      tasks/      tasksSlice.js      Estado +  
thunks de tareas      components/      PrivateRoute.jsx      Protección de rutas  
TaskCard.jsx      Tarjeta de tarea      TaskForm.jsx      Formulario crear/editar  
pages/      LoginPage.jsx      RegisterPage.jsx      DashboardPage.jsx

AdminPage.jsx      ForgotPasswordPage.jsx    Cambio directo de contraseña  
ResetPasswordPage.jsx    Restablecimiento por token    services/  
authService.js    Llamadas API de autenticación    taskService.js    Llamadas API  
de tareas    utils/    axiosConfig.js    Instancia Axios + interceptores

---

## 5. Base de Datos

### Modelo Prisma

El archivo backend/prisma/schema.prisma define dos modelos principales:

```
``prisma model User { id Int @id @default(autoincrement()) email String @unique
password String // Hash bcrypt, nunca texto plano name String role Role
@default(USER) refreshToken String? // Último refreshToken válido emitido
resetToken String? // Token para recuperación de contraseña resetTokenExpiry
DateTime? // Expiración del token de recuperación createdAt DateTime
@default(now()) tasks Task[] // Relación 1:N con Task }
```

```
model Task { id Int @id @default(autoincrement()) title String description String?
status TaskStatus @default(PENDING) priority Priority @default(MEDIUM) dueDate
DateTime? userId Int // FK User.id user User @relation(fields: [userId], references:
[id]) createdAt DateTime @default(now()) updatedAt DateTime @updatedAt }
```

```
enum Role { USER ADMIN } enum TaskStatus { PENDING IN_PROGRESS
COMPLETED } enum Priority { LOW MEDIUM HIGH } ``
```

### Diagrama relacional

|                |                  |              |             | User       | Task          |
|----------------|------------------|--------------|-------------|------------|---------------|
|                |                  |              |             | id (PK)    | < id (PK)     |
| email (UNIQUE) | title            | password     | description | name       | status (enum) |
| role (enum)    | priority (enum)  | refreshToken | dueDate     | resetToken |               |
| userId (FK)    | resetTokenExpiry | createdAt    | createdAt   | updatedAt  |               |

### Comandos de base de datos

```
``bash
```

## Crear y aplicar una migración nueva

```
npx prisma migrate dev --name nombre_migracion
```

# Aplicar migraciones en producción (sin crear archivos nuevos)

```
npx prisma migrate deploy
```

## Insertar datos de prueba

```
npx prisma db seed
```

## Abrir Prisma Studio (interfaz visual)

```
npx prisma studio
```

## Resetear la BD completa (solo desarrollo)

```
npx prisma migrate reset ""
```

---

## 6. Backend — Paso a Paso

### 6.1 Punto de Entrada: server.js

El archivo raíz del backend configura todos los middlewares globales y monta las rutas:

```
``javascript // Necesario cuando la app corre detrás de un proxy (Nginx)
app.set('trust proxy', 1);
```

```
// Seguridad: Helmet agrega headers HTTP protectores app.use(helmet());
```

```
// CORS: solo acepta peticiones del frontend definido en CLIENTURL
app.use(cors({ origin: process.env.CLIENTURL, credentials: true }));
```

```
app.use(express.json()); app.use(cookieParser());
```

```
// Rate limiting: máximo 20 peticiones cada 15 min en /api/auth const authLimiter =
rateLimit({ windowMs: 15 * 60 * 1000, max: 20 }); app.use('/api/auth', authLimiter);
```

```
// Health check — para que el orquestador verifique que el servidor está vivo
app.get('/health', (req, res) => res.json({ status: 'ok' }));
```

```
// Rutas app.use('/api/auth', authRoutes); app.use('/api/tasks', taskRoutes);
app.use('/api/admin', adminRoutes);
```

```
// Captura de errores centralizada (siempre al final) app.use(errorHandler); ``
```

**Por qué este orden importa:** El errorHandler debe ir al final para capturar errores de todas las rutas. El authLimiter debe ir antes de authRoutes para proteger solo esos endpoints. trust proxy permite que express-rate-limit lea la IP real del cliente detrás de Nginx.

---

## 6.2 Configuración de Base de Datos (src/config/db.js)

Exporta una única instancia de PrismaClient para toda la aplicación. Reutilizar la misma instancia evita abrir múltiples conexiones al pool de PostgreSQL.

---

## 6.3 Utilidades JWT (src/utils/jwt.js)

```
``javascript // Access token: vida corta (15 min por defecto) const
generateAccessToken = (payload) => jwt.sign(payload, process.env.JWTSECRET,
{ expiresIn: process.env.JWTACCESSEXPIRESIN || '15m', });
```

```
// Refresh token: vida larga (7 días por defecto), con secreto diferente const
generateRefreshToken = (payload) => jwt.sign(payload,
process.env.JWTREFRESHSECRET, { expiresIn: process.env.JWTREFRESHEXPIRES_IN ||
'7d', }); ``
```

**Diseño de seguridad:** Se usan dos secretos distintos (JWT\_SECRET y JWT\_REFRESH\_SECRET) para que un atacante que comprometa uno de los dos no pueda forjar el otro tipo de token.

El **payload** almacenado en ambos tokens contiene: { id, email, role }.

---

## 6.4 Schemas de Validación con Zod

**src/schemas/authSchemas.js**

```
``javascript const registerSchema = z.object({ name: z.string().min(2).max(100),
email: z.string().email(), password: z.string().min(8).max(100), });
```

```
const loginSchema = z.object({ email: z.string().email(), password:
z.string().min(1), });
```

```
// Cambio directo de contraseña: verifica que el email existe en BD const
forgotPasswordSchema = z.object({ email: z.string().email(), password:
z.string().min(8).max(100), });

// Restablecimiento por token: el token llega por URL desde el email const
resetPasswordSchema = z.object({ token: z.string().min(1), password:
z.string().min(8).max(100), }); ``
```

### **src/schemas/taskSchemas.js**

```
``javascript const createTaskSchema = z.object({ title: z.string().min(1).max(200),
description: z.string().max(1000).optional(), status: z.enum(['PENDING',
'IN_PROGRESS', 'COMPLETED']).optional(), priority: z.enum(['LOW', 'MEDIUM',
'HIGH']).optional(), dueDate: z.string().optional().nullable(), });

// updateTaskSchema hace todos los campos opcionales const updateTaskSchema
= createTaskSchema.partial(); ``
```

**Ventaja de Zod:** La misma lógica de validación se reutiliza en frontend y backend. Si el schema falla, Zod lanza un `ZodError` que el `errorHandler` captura y formatea automáticamente.

---

## **6.5 Middleware de Autenticación (src/middleware/authMiddleware.js)**

```
``javascript const authenticate = (req, res, next) => { const authHeader =
req.headers.authorization; if (!authHeader || !authHeader.startsWith('Bearer '))
return res.status(401).json({ message: 'No token provided' });

const token = authHeader.split(' ')[1]; try { req.user = verifyAccessToken(token); //
Adjunta {id, email, role} a req next(); } catch { res.status(401).json({ message:
'Invalid or expired token' }); } }; ``
```

Este middleware se aplica en todas las rutas que requieren estar autenticado. Adjunta `req.user` con los datos del token para que los controllers puedan usarlo.

---

## **6.6 Middleware de Roles (src/middleware/roleMiddleware.js)**

```
javascript const requireRole = (...roles) => (req, res, next) => { if (!req.user || !
roles.includes(req.user.role)) return res.status(403).json({ message: 'Forbidden:
insufficient permissions' }); next(); };
```



Es una **higher-order function**: acepta uno o más roles y devuelve un middleware. Ejemplo de uso en rutas: `router.get('/users', authenticate, requireRole('ADMIN'), getUsers)`.

---

## 6.7 Manejo Centralizado de Errores (src/middleware/errorHandler.js)

Todos los controllers envían sus errores vía `next(err)` en lugar de responder directamente. Este middleware los captura y formatea la respuesta:

- Errores de Zod (ZodError) **400** con detalles de validación
  - Errores de Prisma (P2002 unique constraint) **409** email duplicado
  - Resto **500** error interno
- 

## 6.8 Controlador de Autenticación (src/controllers/authController.js)

### register

1. Valida el body con `registerSchema.parse()`
2. Verifica que el email no esté registrado
3. Hashea la contraseña con `bcrypt.hash(password, 10)` (10 salt rounds)
4. Crea el usuario en PostgreSQL
5. Devuelve 201 con los datos básicos del usuario (sin contraseña)

### login

1. Valida con `loginSchema.parse()`
2. Busca el usuario por email
3. Compara la contraseña con `bcrypt.compare()`
4. Si válido: genera `accessToken` + `refreshToken`
5. Guarda el `refreshToken` en la BD (columna `refreshToken` del usuario)
6. Envía el `refreshToken` como **cookie HTTP-Only** (inaccesible desde JavaScript)
7. Devuelve el `accessToken` + datos del usuario en el body JSON

### refresh

1. Lee el `refreshToken` de la cookie
2. Verifica su firma con `verifyRefreshToken()`
3. **Doble verificación**: compara el token con el guardado en BD (invalida tokens robados que ya hicieron logout)
4. Genera un nuevo `accessToken` y lo devuelve

## logout

1. Lee el refreshToken de la cookie
2. Pone refreshToken: null en la BD (invalida el token)
3. Limpia la cookie del cliente

## forgotPassword

Flujo de recuperación directa (sin envío de email):

1. Valida el body con forgotPasswordSchema.parse() (email + nueva contraseña)
2. Busca el usuario por email — si no existe devuelve 404
3. Hashea la nueva contraseña con bcrypt.hash()
4. Actualiza la contraseña y pone refreshToken: null (invalida sesiones activas)
5. Devuelve 200 { message: 'Password updated. You can now log in.' }

## resetPassword

Flujo de recuperación por token (diseñado para usarse con email):

1. Valida el body con resetPasswordSchema.parse() (token + nueva contraseña)
2. Busca el usuario cuyo resetToken coincida **y** resetTokenExpiry > now()
3. Si no existe o el token expiró 400
4. Hashea la nueva contraseña
5. Actualiza la contraseña y limpia resetToken, resetTokenExpiry y refreshToken

---

## 6.9 Controlador de Tareas (src/controllers/taskController.js)

### getTasks — Listar con filtros y paginación

```
````javascript const { status, priority, search, page = '1', limit = '10' } = req.query; const where = { userId: req.user.id }; // Solo tareas del usuario autenticado
```

```
if (status) where.status = status; if (priority) where.priority = priority; if (search) { where.OR = [ { title: { contains: search, mode: 'insensitive' } }, { description: { contains: search, mode: 'insensitive' } }, ]; }
```

```
// Ejecuta count y findMany en paralelo para mayor eficiencia const [tasks, total] = await Promise.all([ prisma.task.findMany({ where, skip, take, orderBy: { createdAt: 'desc' } }), prisma.task.count({ where }) ]); ````
```

createTask

1. Valida con createTaskSchema.parse()
2. Convierte dueDate string a objeto Date si viene en el body

3. Crea la tarea asociada al usuario autenticado (userId: req.user.id)

updateTask y deleteTask

Antes de modificar, verifican que el recurso exista y que el usuario sea el **dueño** o tenga rol **ADMIN**. Esto previene que un usuario edite tareas de otro.

exportCSV

Obtiene todas las tareas del usuario y las convierte a formato CSV usando csvExporter.js, luego las envía con los headers Content-Type: text/csv y Content-Disposition: attachment.

6.10 Controlador de Administración (src/controllers/adminController.js)

Requiere authenticate + requireRole('ADMIN') en todas sus rutas.

| Función | Descripción |
|----------------|--|
| getUsers | Devuelve todos los usuarios sin exponer contraseñas (select explícito) |
| getUserTasks | Devuelve todas las tareas de un usuario por su ID |
| updateUserRole | Cambia el rol de un usuario entre USER y ADMIN |

6.11 Rutas

src/routes/authRoutes.js

POST /register register POST /login login POST /refresh refresh POST /logout
logout POST /forgot-password forgotPassword POST /reset-password
resetPassword

src/routes/taskRoutes.js

Todas protegidas con authenticate: GET / getTasks POST / createTask PUT /:id
updateTask DELETE /:id deleteTask GET /export exportCSV

src/routes/adminRoutes.js

Todas protegidas con authenticate + requireRole('ADMIN'): GET /users getUsers
GET /users/:id/tasks getUserTasks PATCH /users/:id/role updateUserRole

7. Frontend — Paso a Paso

7.1 Entrada de la Aplicación

src/main.jsx envuelve la app con `<Provider store={store}>` (Redux) y `<BrowserRouter>` (React Router), habilitando el estado global y el enrutamiento en toda la aplicación.

7.2 Redux Store (src/app/store.js)

Combina dos slices:

```
javascript { auth: authReducer, // { user, accessToken, loading, error } tasks: tasksReducer, // { items, total, page, loading, error } }
```

El accessToken vive **en memoria** (Redux), nunca en localStorage. Esto lo protege de ataques XSS. Si el usuario recarga la página, el token se pierde pero el interceptor de Axios lo renueva automáticamente usando la cookie HTTP-Only.

7.3 Axios con Interceptores (src/utils/axiosConfig.js)

Este archivo es el corazón de la comunicación con el backend.

Interceptor de request

Antes de enviar cualquier petición, adjunta el accessToken del store de Redux:

```
javascript api.interceptors.request.use((config) => { const token = store.getState().auth.accessToken; if (token) config.headers.Authorization = `Bearer ${token}`; return config; });
```

Interceptor de response — Renovación automática del token

Cuando el servidor responde con **401** (token expirado):

1. Si ya hay un refresh en curso (isRefreshing = true), encola la petición fallida en failedQueue.
2. Si no hay refresh en curso, inicia el proceso de renovación llamando a POST / auth/refresh.
3. El nuevo accessToken se guarda en Redux con setAccessToken.
4. Se resuelven todas las peticiones en cola con el nuevo token.
5. Si el refresh falla, hace logout() y redirige al login.

El resultado: el usuario nunca nota que su token expiró — la petición original se reintenta de forma transparente.

7.4 Slice de Autenticación (src/features/auth/authSlice.js)

Define el estado de autenticación y los **async thunks**:

| Thunk | Acción |
|------------------------|--|
| loginUser(credentials) | POST /auth/login guarda user + accessToken |
| registerUser(data) | POST /auth/register |
| logoutUser() | POST /auth/logout limpia el estado |

Estados de carga: cada thunk maneja pending fulfilled rejected para actualizar loading y error en el store.

Reducers síncronos:

- setAccessToken(token) — usado por el interceptor de Axios al renovar el token
 - logout() — limpia user, accessToken y error del estado
-

7.5 Slice de Tareas (src/features/tasks/tasksSlice.js)

| Thunk | Endpoint |
|--------------------------|---|
| fetchTasks(params) | GET /tasks?status=&priority=&search=&page=&limit= |
| createTask(data) | POST /tasks |
| updateTask({ id, data }) | PUT /tasks/:id |
| deleteTask(id) | DELETE /tasks/:id |

Actualizaciones del estado:

- createTask.fulfilled agrega la nueva tarea al inicio del array con unshift
 - updateTask.fulfilled reemplaza la tarea por índice
 - deleteTask.fulfilled filtra la tarea del array y decremента total
-

7.6 Componente PrivateRoute (src/components/PrivateRoute.jsx)

```
```javascript export default function PrivateRoute({ requiredRole }) { const { user, accessToken } = useSelector((state) => state.auth);
```

```
if (!accessToken) return ; if (requiredRole && user?.role !== requiredRole) return ;
return ; } ```
```

Protege las rutas en dos niveles:

1. **Sin sesión** (!accessToken) redirige a /login
  2. **Sin rol suficiente** (role !== requiredRole) redirige a /dashboard
- 

## 7.7 Páginas

### LoginPage.jsx

Formulario de login con React Hook Form + Zod. Despacha loginUser() y redirige a /dashboard al éxito.

### RegisterPage.jsx

Formulario de registro con validaciones Zod (nombre mínimo 2 chars, email válido, contraseña mínimo 8 chars). Redirige a /login al completarse.

### DashboardPage.jsx

Vista principal del usuario:

- Carga las tareas con fetchTasks() al montar el componente
- Barra de filtros (estado, prioridad, búsqueda)
- Lista de <TaskCard> con opciones de editar y eliminar
- Botón para abrir <TaskForm> y crear nueva tarea
- Botón de exportar CSV

### AdminPage.jsx

Panel exclusivo para administradores:

- Lista de todos los usuarios del sistema
- Al seleccionar un usuario, muestra sus tareas
- Botón para cambiar el rol entre USER/ADMIN

### ForgotPasswordPage.jsx

Formulario de recuperación directa de contraseña:

- Campos: email, nueva contraseña y confirmación de contraseña
- Validación Zod: contraseñas deben coincidir, mínimo 8 caracteres
- Llama a POST /api/auth/forgot-password con { email, password }
- Al éxito redirige a /login con mensaje de confirmación

## ResetPasswordPage.jsx

Formulario de restablecimiento por token (flujo de email):

- Lee el token de la URL
  - Campos: nueva contraseña y confirmación
  - Llama a POST /api/auth/reset-password con { token, password }
  - Al éxito redirige a /login
- 

## 8. Flujo de Autenticación JWT

### Registro y primer login

| Usuario Frontend (React) | Backend (Express)     | Base de Datos  | Formulario |
|--------------------------|-----------------------|----------------|------------|
| POST /auth/register      | INSERT User           | { id,          |            |
| name, email }            | 201 Created           | Redirige login |            |
| Credenciales             | POST /auth/login      | SELECT User    |            |
| bcrypt.compare()         | UPDATE refreshToken   | { accessToken, |            |
| user }                   | + Cookie refreshToken | Dashboard      |            |

### Renovación automática del token

| Frontend Axios Interceptor | Backend          |                  |
|----------------------------|------------------|------------------|
| GET /tasks                 | GET /tasks       |                  |
|                            | 401 Unauthorized | (token expirado) |
| POST /auth/refresh         | (usa cookie)     | { accessToken }  |
| tasks (retry)              | Tareas recibidas | 200 OK + tasks   |

### Recuperación directa de contraseña

| Usuario Frontend | Backend       | Base de Datos     | Formulario      |             |
|------------------|---------------|-------------------|-----------------|-------------|
| POST             | nueva pass    | /auth/            | forgot-password | email +     |
|                  | bcrypt.hash() | UPDATE password,  |                 | SELECT User |
| 200 OK           | Login page    | refreshToken=null |                 |             |

---

## 9. API Reference

### Health

| Método | Ruta | Auth | Respuesta |
|--------|------|------|-----------|
|--------|------|------|-----------|

|     |         |   |                      |
|-----|---------|---|----------------------|
| GET | /health | — | 200 { status: 'ok' } |
|-----|---------|---|----------------------|

## Autenticación

| Método | Ruta                      | Auth   | Body                    | Respuesta                        |
|--------|---------------------------|--------|-------------------------|----------------------------------|
| POST   | /api/auth/register        | —      | {name, email, password} | 201 {id, name, email, role}      |
| POST   | /api/auth/login           | —      | {email, password}       | 200 {accessToken, user} + cookie |
| POST   | /api/auth/refresh         | Cookie | —                       | 200 {accessToken}                |
| POST   | /api/auth/logout          | Cookie | —                       | 200 {message}                    |
| POST   | /api/auth/forgot-password | —      | {email, password}       | 200 {message}                    |
| POST   | /api/auth/reset-password  | —      | {token, password}       | 200 {message}                    |

## Tareas

| Método | Ruta              | Auth   | Descripción                          |
|--------|-------------------|--------|--------------------------------------|
| GET    | /api/tasks        | Bearer | Listar tareas con filtros opcionales |
| POST   | /api/tasks        | Bearer | Crear nueva tarea                    |
| PUT    | /api/tasks/:id    | Bearer | Actualizar tarea (dueño o admin)     |
| DELETE | /api/tasks/:id    | Bearer | Eliminar tarea (dueño o admin)       |
| GET    | /api/tasks/export | Bearer | Descargar tareas como CSV            |

### Query params para GET /api/tasks

| Parámetro | Tipo   | Ejemplo                         |
|-----------|--------|---------------------------------|
| status    | enum   | PENDING, IN_PROGRESS, COMPLETED |
| priority  | enum   | LOW, MEDIUM, HIGH               |
| search    | string | implementar login               |
| page      | number | 1 (default)                     |
| limit     | number | 10 (default)                    |

## Administración (solo ADMIN)

| Método | Ruta                       | Descripción                 |
|--------|----------------------------|-----------------------------|
| GET    | /api/admin/users           | Listar todos los usuarios   |
| GET    | /api/admin/users/:id/tasks | Tareas de un usuario        |
| PATCH  | /api/admin/users/:id/role  | Cambiar rol {role: "ADMIN"} |



## Ejemplos

### POST /api/auth/login

```
```json // Request { "email": "usuario@ejemplo.com", "password": "MiPassword123" }

// Response 200 { "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...", "user":
{ "id": 1, "name": "Euclides", "email": "usuario@ejemplo.com", "role": "USER" } } ```
```

POST /api/auth/forgot-password

```
```json // Request { "email": "usuario@ejemplo.com", "password":
"NuevaPassword123" }

// Response 200 { "message": "Password updated. You can now log in." }

// Response 404 (email no encontrado) { "message": "No account found with that
email." } ```
```

### POST /api/tasks

```
```json // Request (Authorization: Bearer ) { "title": "Implementar refresh tokens",
"description": "Agregar endpoint POST /auth/refresh", "status": "IN_PROGRESS",
"priority": "HIGH", "dueDate": "2026-06-01" }

// Response 201 { "id": 42, "title": "Implementar refresh tokens", "status":
"IN_PROGRESS", "priority": "HIGH", "dueDate": "2026-06-01T00:00:00.000Z",
"userId": 1, "createdAt": "2026-05-18T10:00:00.000Z" } ```
```

10. Variables de Entorno

Raíz del proyecto — .env (Docker Compose)

```
```env
```

## PostgreSQL

```
POSTGRESDB=taskmanagerdb POSTGRESUSER=postgres
POSTGRESPASSWORD=tupassword_seguro
```

# JWT — usar strings de 64+ caracteres generados aleatoriamente

*JWTSECRET=stringmuylargoyaleatorioparaaccesstokens*  
*JWTREFRESHSECRET=otrostringcompletamentedistintopararefresh*  
*JWTACCESSEXPIRESIN=15m JWTREFRESHEXPIRES\_IN=7d*

## CORS — URL exacta del frontend

*CLIENT\_URL=https://tu-dominio.com*

## URL del backend (se inyecta en el build del frontend)

*VITEAPIURL=https://tu-dominio.com/api*

## Email (para recuperación de contraseña por token)

*EMAILHOST=smtp.gmail.com EMAILPORT=587 EMAILUSER=tu@gmail.com*  
*EMAILPASS=tuapppassworddegmail ``*

## Backend — backend/.env (desarrollo local)

*``env*

## Base de datos

*DATABASE\_URL="postgresql://usuario:password@localhost:5432/taskmanagerdb"*

# JWT — usar strings largos y aleatorios (mínimo 32 caracteres)

```
JWTSECRET="stringmuylargoyaleatorioparaaccesstokens"
JWTREFRESHSECRET="otrostringcompletamentedistintopararefresh"
JWTACCESSEXPIRESIN="15m" JWTREFRESHEXPIRES_IN="7d"
```

## Servidor

PORT=3000 NODE\_ENV=development # Cambiar a "production" en deploy

## CORS — URL exacta del frontend (incluir protocolo y puerto)

CLIENT\_URL="http://localhost:5173"

## Email (opcional — para recuperación de contraseña)

```
EMAILHOST="smtp.gmail.com" EMAILPORT=587 EMAILUSER="tu@gmail.com"
EMAILPASS="tuapppassworddegmail" ``
```

## Frontend — frontend/.env (desarrollo local)

env VITE\_API\_URL="http://localhost:3000/api"

**Importante:** Nunca incluir archivos .env en git. El .gitignore ya los excluye.

---

# 11. Instalación y Puesta en Marcha

## Requisitos previos

- Node.js v20 o superior (node --version)
- PostgreSQL v14 o superior corriendo localmente (solo sin Docker)
- npm v9 o superior

- Docker y Docker Compose (para despliegue en contenedores)

## **Paso 1 — Clonar el repositorio**

```
bash git clone https://github.com/tu-usuario/task-manager-jwt.git cd task-manager-jwt
```

## **Paso 2 — Configurar el backend**

```
``bash cd backend npm install cp .env.example .env
```

# **Editar .env con tus credenciales de PostgreSQL y secretos JWT**

```
...
```

## **Paso 3 — Crear la base de datos y correr migraciones**

```
``bash
```

# **Crear la base de datos en PostgreSQL:**

```
psql -U postgres -c "CREATE DATABASE taskmanagerdb;"
```

# **Aplicar las migraciones con Prisma:**

```
npm run prisma migrate dev
```

# **(Opcional) Cargar datos de prueba:**

```
npm run prisma db seed ``
```

## **Paso 4 — Iniciar el backend**

```
``bash npm run dev
```

# Servidor corriendo en http://localhost:3000

...

## Paso 5 — Configurar el frontend

```
``bash cd ../frontend npm install cp .env.example .env
```

# Verificar que VITEAPIURL apunte al backend correcto

...

## Paso 6 — Iniciar el frontend

```
``bash npm run dev
```

# Frontend en http://localhost:5173

...

## Paso 7 — Build para producción

```
``bash cd frontend npm run build
```

# Los archivos estáticos quedan en frontend/dist/

...

---

## 12. Docker — Contenedorización

El proyecto está completamente contenedorizado con Docker Compose. Cada servicio tiene su propio Dockerfile optimizado para producción.

## **docker-compose.yml**

```
``yaml services: task-db: image: postgres:16-alpine containername: task-manager-
db restart: unless-stopped environment: POSTGRESDB: ${POSTGRESDB}
POSTGRESUSER: ${POSTGRESUSER} POSTGRESPASSWORD: ${POSTGRESPASSWORD}
volumes: - postgresdata:/var/lib/postgresql/data networks: - taskinternalnet
healthcheck: test: ["CMD-SHELL", "pgisready -U ${POSTGRESUSER} -d $$
{POSTGRES_DB}"] interval: 5s timeout: 5s retries: 10
```

```
task-backend: build: context: ./backend containername: task-manager-backend
restart: unless-stopped dependson: task-db: condition: servicehealthy # Espera a que
PostgreSQL esté listo environment: DATABASEURL: postgresql://${POSTGRESUSER}:$
{POSTGRESPASSWORD}@task-db:5432/${POSTGRESDB} # ... resto de variables
networks: - proxynetwork # Expuesto al proxy externo - taskinternalnet # Conectado
a la BD de forma privada
```

```
task-frontend: build: context: ./frontend args: VITEAPIURL: ${VITEAPIURL} #
Inyectada en tiempo de build containername: task-manager-frontend restart: unless-
stopped dependson: - task-backend networks: - proxy_network
```

```
networks: proxynetwork: external: true # Red compartida con Nginx Proxy Manager
taskinternal_net: driver: bridge # Red privada exclusiva de este proyecto
```

```
volumes: postgres_data: ``
```

## **Backend — Dockerfile**

```
``dockerfile FROM node:20-alpine
```

# **Dependencias del sistema que necesita Prisma en Alpine**

```
RUN apk add --no-cache openssl libc6-compat
```

```
ENV NODE_ENV=production
```

```
WORKDIR /app
```

```
COPY package*.json ./ RUN npm ci
```

```
COPY . .
```

```
RUN npx prisma generate \ && addgroup -S appgroup \ && adduser -S appuser -G
appgroup \ && chown -R appuser:appgroup /app \ && chmod +x entrypoint.sh
```

```
USER appuser EXPOSE 3000 ENTRYPOINT ["/entrypoint.sh"] ``
```

## Backend — entrypoint.sh

```
``sh
```

## !/bin/sh

```
set -e npx prisma migrate deploy # Aplica migraciones pendientes al arrancar exec
node server.js ``
```

**Por qué es importante:** Al arrancar el contenedor, las migraciones se aplican automáticamente antes de que el servidor acepte peticiones. Esto garantiza que la BD esté siempre actualizada sin intervención manual.

## Frontend — Dockerfile (build multietapa)

```
``dockerfile
```

## Etapa 1: Compilación con Node

```
FROM node:20-alpine AS builder WORKDIR /app COPY package*.json ./ RUN npm ci
--legacy-peer-deps COPY . .
```

## La URL del backend se inyecta en tiempo de build para que Vite la embeba

```
ARG VITEAPIURL ENV VITEAPIURL=$VITEAPIURL
```

```
RUN npm run build
```

## Etapa 2: Servidor Nginx mínimo para los archivos estáticos

```
FROM nginx:alpine COPY --from=builder /app/dist /usr/share/nginx/html COPY
nginx.conf /etc/nginx/conf.d/default.conf EXPOSE 80 CMD ["nginx", "-g", "daemon
off;"] ``
```

## Frontend — nginx.conf

```
``nginx server { listen 80; server_name _;

root /usr/share/nginx/html;
index index.html;

SPA fallback — rutas no encontradas devuelven index.html (React Router)
location / {
 try_files $uri $uri/ /index.html;
}

Caché agresivo para assets con hash en el nombre (generados por Vite)
location ~* \.(js|css|png|jpg|jpeg|gif|ico|svg|woff|woff2|ttf|eot)$ {
 expires 1y;
 add_header Cache-Control "public, immutable";
}

} ``
```

**Diseño de la imagen:** La imagen de producción del frontend solo contiene los archivos estáticos compilados y Nginx — sin Node.js, sin código fuente, sin node\_modules. Tamaño mínimo, superficie de ataque mínima.

---

## 13. Tests

Los tests usan **Jest** + **Supertest** y se encuentran en backend/tests/.

```
bash cd backend npm test
```

La configuración de Jest está en package.json:

```
json "jest": { "testEnvironment": "node", "testRegex": "tests/.*.test\\.js$" }
```

La flag `--runInBand` hace que los tests se ejecuten secuencialmente (no en paralelo), lo cual es necesario porque comparten la misma base de datos de test.

### tests/auth.test.js — Casos de prueba

| Test                                      | Código esperado   |
|-------------------------------------------|-------------------|
| POST /auth/register — registro exitoso    | 201               |
| POST /auth/register — email duplicado     | 409               |
| POST /auth/login — credenciales correctas | 200 + accessToken |
| POST /auth/login — password incorrecto    | 401               |



| Test                                 | Código esperado         |
|--------------------------------------|-------------------------|
| POST /auth/login — usuario no existe | 401                     |
| POST /auth/refresh — cookie válida   | 200 + nuevo accessToken |
| POST /auth/logout — limpia cookie    | 200                     |

### tests/tasks.test.js — Casos de prueba

| Test                                                    | Código esperado      |
|---------------------------------------------------------|----------------------|
| GET /tasks — sin token                                  | 401                  |
| GET /tasks — con token válido                           | 200 + lista paginada |
| POST /tasks — crea tarea correctamente                  | 201                  |
| POST /tasks — validación falla                          | 400                  |
| PUT /tasks/:id — dueño puede editar                     | 200                  |
| DELETE /tasks/:id — usuario no puede borrar tarea ajena | 403                  |

---

## 14. Deploy con Docker y Nginx Proxy Manager

### Arquitectura de red en producción

Internet    Nginx Proxy Manager (red: proxy\_network — externa)    task-frontend : 80 (Nginx Alpine + build estático)    task-backend :3000 (Node 20 Alpine)  
 task\_internal\_net (bridge privada)    task-db :5432 (PostgreSQL 16 — no expuesta)

La BD nunca está en la red proxy\_network, solo en la red interna privada. Solo el backend puede hablar con ella.

### Paso 1 — Crear la red externa (solo una vez)

```
bash docker network create proxy_network
```

### Paso 2 — Configurar las variables de entorno

```
```bash cp .env.example .env
```

Editar con:

- **POSTGRES_PASSWORD** fuerte
- **JWTSECRET** y **JWTREFRESH_SECRET** de 64+ caracteres
- **CLIENTURL** y **VITEAPI_URL** con el dominio real

...

Paso 3 — Construir e iniciar los servicios

```
bash docker compose up -d --build
```

El backend esperará a que PostgreSQL pase su healthcheck antes de arrancar. Luego ejecutará prisma migrate deploy automáticamente.

Paso 4 — Verificar que los servicios están corriendo

```
bash docker compose ps docker compose logs -f task-backend
```

Paso 5 — Configurar Nginx Proxy Manager

Crear entradas de Proxy Host para que el tráfico llegue a los contenedores:

| Dominio | Destino | Puerto |
|--------------------|----------------|---------------|
| tu-dominio.com | task-frontend | 80 |
| api.tu-dominio.com | task-backend | 3000 |

O usar un único dominio con path-based routing:

- /api/ task-backend:3000
- / task-frontend:80

Activar SSL con Let's Encrypt en cada entrada (checkbox "Request a new SSL Certificate").

Actualización del servidor

```
bash git pull docker compose up -d --build
```

Las migraciones de BD se aplican automáticamente al reiniciar el backend.

Gestión del volumen de datos

```
```bash
```

## Ver logs

```
docker compose logs -f
```

## Parar sin borrar datos

```
docker compose down
```

## Parar y borrar todos los datos de la BD

```
docker compose down -v ```
```

---

## 15. Seguridad

### Medidas implementadas

| Medida                  | Implementación                           | Protege contra                     |
|-------------------------|------------------------------------------|------------------------------------|
| Hash de contraseñas     | bcrypt con 10 salt rounds                | Robo de base de datos              |
| JWT de corta duración   | Access token: 15 minutos                 | Robo de tokens                     |
| Refresh token HTTP-Only | Cookie inaccesible desde JS              | XSS                                |
| SameSite: Strict        | Cookie no enviada en requests cross-site | CSRF                               |
| Rate limiting           | 20 requests / 15 min en /api/auth        | Fuerza bruta / credential stuffing |
| Helmet                  | Headers HTTP seguros                     | Inyección de headers, clickjacking |
| Validación Zod          | En backend antes de cada operación       | Datos malformados, inyección       |

| Medida                               | Implementación                            | Protege contra                             |
|--------------------------------------|-------------------------------------------|--------------------------------------------|
| CORS restringido                     | Solo acepta el CLIENT_URL configurado     | Requests de orígenes no autorizados        |
| Autorización por recurso             | Verifica userId antes de editar/borrar    | Escalación horizontal de privilegios       |
| Doble verificación del refresh token | Compara con el token almacenado en BD     | Reuso de tokens tras logout                |
| Trust proxy                          | app.set('trust proxy', 1)                 | Rate limit correcto detrás de Nginx        |
| Usuario sin privilegios en Docker    | Corre como appuser (no root)              | Escalación de privilegios en el contenedor |
| BD no expuesta en red pública        | Solo en task_internal_net                 | Acceso directo a la base de datos          |
| Build multietapa (frontend)          | Imagen final sin Node.js ni código fuente | Exposición del código fuente               |

## Importante en producción

1. NODE\_ENV=production está forzado en docker-compose.yml activa secure: true en la cookie del refresh token (solo HTTPS)
2. Usar HTTPS — el secure: true en la cookie lo requiere. Nginx Proxy Manager lo gestiona automáticamente con Let's Encrypt
3. Cambiar los secretos JWT por strings de 64+ caracteres generados aleatoriamente
4. Configurar DATABASE\_URL apuntando al contenedor task-db (ya configurado en docker-compose.yml)

---

## Autor

### Euclides Marín

Proyecto de práctica — Stack PERN con autenticación JWT completa y despliegue Docker.